Information Technology Course
Module Software Engineering
by Damir Dobric / Andreas Pech

ML 24/25-07 Extract and Evaluate Embeddings from SDR

Amina Aktar Rubi
amina.rubi@stud.fra-uas.de

Al Amin Ul Islam
al.islam@stud.fra-uas.de

Md Mahabubur Rashid Khan
mahabubur.khan@stud.fra-uas.de

Abstract*—Hierarchical Temporal Memory (HTM) is a framework with biological inspiration that simulates the neocortex's capacity to identify patterns in temporal sequences. This study explores the integration of HTM with the BERT language model for semantic similarity analysis in long sequential text data. Using the NeocortexAPI framework, we preprocess input sequences through BERT-based tokenization, followed by HTM-based encoding with predefined spatial and temporal parameters. The encoded representations are processed through Spatial Pooling (SP) and Temporal Memory (TM) layers to capture sequential patterns. Semantic similarity is evaluated using cosine and Euclidean distance metrics. Experimental results demonstrate that HTM layers effectively learn sequential dependencies, improving semantic similarity detection compared to traditional methods. The findings suggest that HTM-enhanced models can better interpret contextual relationships in extended text sequences, offering potential applications in natural language understanding and sequence analysis tasks.

Keywords—Hierarchical Temporal Memory (HTM), Sparse Distributed Representations (SDRs), Semantic Similarity, Spatial Pooling, Word Tokenization

I. INTRODUCTION

Hierarchical Temporal Memory (HTM) is a biologically inspired machine learning framework that mimics the neocortex's ability to recognize patterns in temporal sequences. This innovative algorithm is built upon the principles of temporal hierarchy and sparsity, making it a groundbreaking advancement in the field of machine learning. By leveraging these principles, HTM offers a robust framework for analyzing and modeling sequential data, emphasizing the dynamic interplay between spatial and temporal patterns. Multi-sequence Learning has significant industrial applications, such as pattern recognition in music sequences, anomaly detection in sensor data, and predictive analytics in time-series forecasting.

At the core of HTM lies the Spatial Pooler (SP), a crucial computational mechanism responsible for converting raw input data into Sparse Distributed Representations (SDRs). This transformation process is not merely a mathematical construct but is designed to mirror the way the human brain encodes and processes information. By maintaining sparsity and distributed activation, the Spatial Pooler enables efficient pattern recognition, memory formation, and information retrieval. Similar to the way the neocortex organizes and interprets sensory inputs, the SP ensures that learned representations are both stable and adaptable, facilitating the recognition of recurring patterns in a highly efficient manner.

This project explores the application of HTM and the Neocortex API [1] in Multi-sequence Learning, a process that enables machines to learn and predict multiple sequences. Here, sequences of tokens are converted into SDRs, which serve as unique encodings for different elements in a sequence. The SDRs capture the relationships between sequential elements, preserving structural information. The primary goal is to compare the similarity of SDRs using embedding-based techniques, specifically cosine and Euclidean similarity functions. By analyzing these similarities, we can assess how well different sequences relate to each other in a high-dimensional space.

The project follows a structured approach, beginning with tokenization of input sequences using BERT (Bidirectional Encoder Representations from Transformers) tokenizer [2], conversion of tokens into token-IDs and later converting them into SDRs, and lastly performed subsequent similarity analysis. By leveraging HTM's capabilities, this study aims to enhance sequence recognition through robust similarity measurements, contributing to advancements in machine learning and artificial intelligence. The detailed discussion about the approach is described in the following sections in this paper.

II. LITERATURE REVIEW

A. Hierarchical Temporal Memory (HTM):

Unlike traditional machine learning approaches that rely on statistical learning, Hierarchical Temporal Memory (HTM) mimics cortical processing by employing continuous learning, sparse representations, and sequence memory [2]. It is particularly effective in applications such as anomaly detection, time-series forecasting, and cognitive computing. The key components of HTM include Sparse Distributed Representations (SDRs), the Spatial Pooler, and the Temporal Memory, which together form an efficient system for processing temporal data. HTM's online learning ability makes it well-suited for real-time data streams, setting it apart from deep learning approaches that typically require batch processing and large labeled datasets.

B. Sparse Distributed Representations (SDRs):

Sparse Distributed Representations (SDRs) are binary vectors with a fixed number of active bits (1s) distributed sparsely across a high-dimensional space [3]. SDRs provide a biologically plausible way to encode and store information, ensuring robustness to noise, efficient pattern recognition, and fault tolerance. One of the key properties of SDRs is their ability to maintain semantic similarity, where similar inputs result in overlapping SDRs. This allows for efficient sequence learning and generalization in HTM systems. Furthermore, SDRs provide a unique advantage over traditional dense representations by enabling high-capacity storage and fast retrieval while preserving meaningful data relationships.

C. Encoders:

Encoders in HTM play a crucial role in converting raw input data into Sparse Distributed Representations (SDRs), making them suitable for processing by the HTM network. Different types of encoders exist depending on the data type being processed. For example:

- *Scalar Encoders* convert numerical values into binary SDRs while preserving ordering and scale.
- *Category Encoders* assign distinct SDRs to categorical inputs, ensuring uniqueness.
- *Date-Time Encoders* encode temporal information, allowing HTM to process periodic data efficiently.

Since HTM relies on high-quality input representations, the effectiveness of an encoder significantly influences the overall performance of the system. Poor encoding can lead to inaccurate sequence learning and decreased prediction accuracy.

In this project we are using Scalar Encoder to convert our input into SDRs

D. Spatial Pooler:

The Spatial Pooler is a fundamental component of HTM responsible for generating stable and sparse representations of input patterns. It ensures that similar inputs produce overlapping SDRs while preserving sparsity, enabling the model to generalize from learned patterns. The Spatial Pooler achieves this by selecting a fixed number of active columns from a larger set based on input activity levels. It incorporates mechanisms such as synaptic permanence and inhibition to reinforce frequently occurring patterns while adapting to novel inputs. This process allows HTM to identify recurring structures in data streams and supports continuous learning.

E. Temporal Memory:

Temporal Memory extends HTM's capabilities by learning and recalling sequences of patterns over time. It models the ability of cortical neurons to form connections based on temporal dependencies, allowing the system to predict future inputs based on past observations. Temporal Memory achieves this through active dendrites, which enable neurons to recognize context-dependent patterns. This mechanism is crucial for applications requiring sequence prediction, such as speech recognition, anomaly detection, and time-series forecasting. Unlike traditional recurrent neural networks (RNNs), HTM's Temporal Memory does not require backpropagation or fixed training phases, making it suitable for real-time learning applications.

F. BertTokenizer:

The BERT tokenizer is an essential component of the BERT (Bidirectional Encoder Representations from Transformers) model, designed to efficiently preprocess textual data for deep learning applications. It employs **WordPiece tokenization** [4], which splits words into subwords based on frequency-based vocabulary learning. This approach enables the model to handle out-of-vocabulary words and capture meaningful subword representations. The BERT tokenizer plays a critical role in transforming raw text into numerical inputs for transformer-based architectures, preserving syntactic and semantic relationships. It has been widely adopted in natural language processing (NLP) tasks, including sentiment analysis, machine translation, and question answering. The tokenization process ensures that complex linguistic structures are broken down into manageable units, improving the model's ability to understand contextual meanings.

III. METHODOLOGY

A. Overview

The methodology for this project is structured into several key steps, each designed to achieve specific objectives in the sequence learning and similarity comparison process. The stepwise workflow of this project is as follows:

1. The input text file is first converted into token-IDs.
2. Token-ID to SDRs: The generated token-IDs are transformed into binary SDR representations.
3. Text Subsequence Selection for Comparison: Two distinct text subsequences are extracted from the input for similarity analysis.
4. Cosine Similarity Computation: Cosine similarity is calculated between the SDR representations of the selected sequences.

In the second step, to analyze and compare sequence learning performance, SDRs were computed using two distinct approaches:

- **using a Scalar Encoder**. The Token IDs were transformed into binary representations using a scalar encoder. Throughout this documentation, we will refer to the binary outputs generated by this scalar encoder as **Direct SDRs or Basic SDRs** for simplicity and consistency. Cosine and Euclidean similarity were computed directly from these binary representations.

-Another one is using **HTM Layer**. The binary sequences initially encoded by the scalar encoder were further processed through an HTM layer to capture temporal and spatial patterns. The resulting output, which reflects the learned temporal context and structure, is referred to throughout this documentation as **HTM SDRs**.

The HTM Layers involved in generating **HTM SDRs** include:

- Spatial Pooler (SP) for feature extraction.
- Temporal Memory (TM) for learning sequential patterns.

Together, these components work to transform basic encoded inputs into HTM SDRs, were then used for cosine similarity computation.

By comparing similarity calculations from both methods, this study aims to: Examine how **HTM-based sequence learning** influences similarity results. And identify the differences between **directly encoded** and **HTM-processed** sequences.

The following figure provides a graphical representation of the overall workflow:

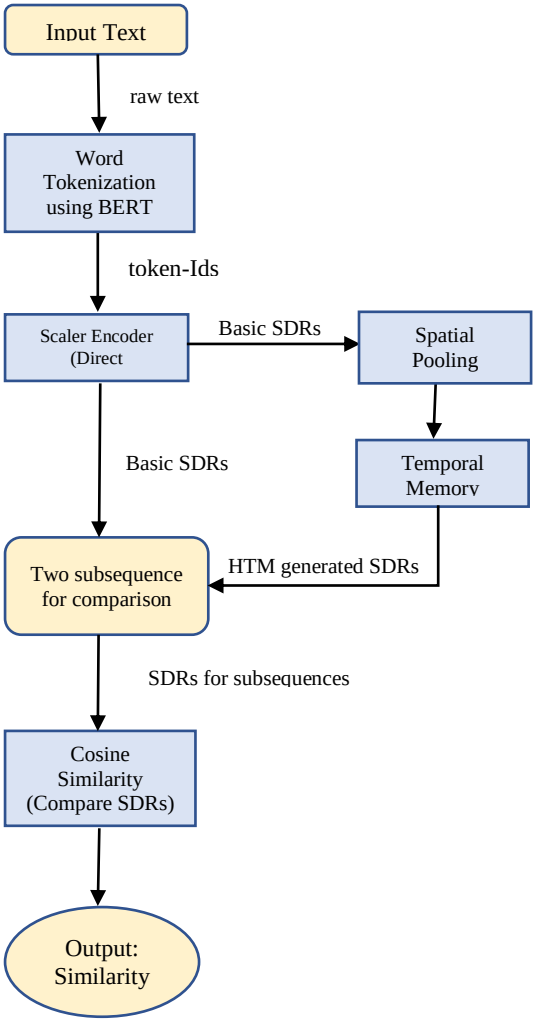

Figure 1: Graphical Representation of Overall Workflow

B. Word Tokenization:

A text input (sourceText) containing more than 100 words (to be specific, 116 words) is selected as the primary dataset. This input serves as the source for generating SDRs.

The sourceText is tokenized into individual words using a transformer-based tokenization function (BertTokenizer) [5]. This step ensures that the input is broken down into manageable units. Each unique word is assigned a token id (integer numbers) for further processing. The reason behind using this model that it splits text into **subwords**, **words**, and **word pieces**, allowing it to handle rare or unknown words effectively. This model first converts all text to **lowercase** and ignores case differences as case difference or maintaining capitalization is not important for calculating sematic similarity. The figure given below depicts how word tokenization using “bert-base-uncased” works:

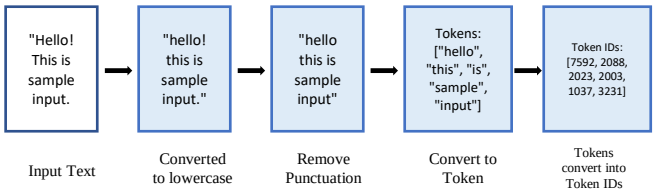

Figure 2: Conversion of text inputs into token-IDs

Following code snippet illustrates how to tokenize the (sourceText) and assign id to each unique token:

```
from transformers import BertTokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
input_text = sys.argv[1]
tokens = tokenizer.tokenize(input_text)
token_ids = tokenizer.convert_tokens_to_ids(tokens)
```

C. SDR Generation for sourceText:

Each token id is converted into an SDR using the Scaler Encoder. An SDR is a high-dimensional binary vector with a fixed number of bits, where a small subset of bits is set to "1" to represent the word. The generated SDRs ensure that similar words or subsequences produce SDRs with overlapping patterns.

Following code snippet illustrates the Scaler Encoder parameters:

```
Dictionary<string, object> settings = new
Dictionary<string, object>()
 {
    { "W", 15},
    { "N", inputBits},
    { "Radius", -1.0},
    { "MinVal", 0.0},
    { "Periodic", false},
    { "Name", "scalar"},
    { "ClipInput", false},
    { "MaxVal", max}
 };
```

W is the bit population used to represent a single input value. N is the total number of bits the encoder will use to encode the input [3]. To determine the maximum value MaxVal from a dictionary containing key-value pairs, where the values are represented as lists of numbers, we utilized the **Max()** method from the **System.Linq** namespace. This approach involves flattening all the lists within the dictionary

into a single sequence and then identifying the highest value. The following code snippet demonstrates this process:

```
double max = sequences.Values.SelectMany(list => list).Max();
```

All the generated SDRs using Scaler encoder are stored in a separate file for calculating similarity without learning the sequence encoder using HTM layer. The SDRs values were saved to a text file and managing SDR data in a dictionary. First, it converts the result array into two string representations, which contains the full SDR binary vector, and sdrText, which stores only the indices of active bits (values equal to 1). The formatted SDR data is then appended to a file without overwriting previous entries. After writing to the file, the code checks whether the input key already exists in the sdrs dictionary. If the key is **not present**, it adds the input sequence as a key with its corresponding SDR values as the dictionary value. If the key **already exists**, a message is printed to the console stating that the input has been skipped to prevent duplicate entries. This approach ensures efficient SDR data management while preventing redundant storage. The following figure depicts the process:

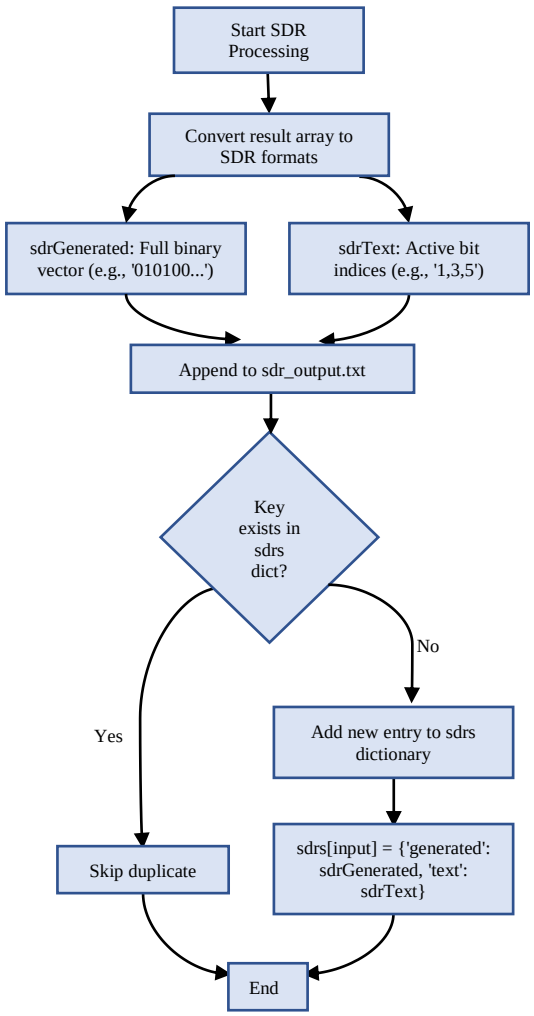

Figure 3: Storing Generated SDRs (from Scaler Encoder) for processing

D. SDR mapping:

A second text input (subsequenceText), which is a subsequence of the sourceText, is tokenized and processed. The SDR indexes for subsequenceText is derived from the stored SDRs of sourceText.

A third text input (comparisonText), which may or may not be a subsequence of sourceText, is also tokenized following the same process as subsequenceText. If the tokens in comparisonText exist in sourceText, their corresponding SDRs are retrieved. If not, a warning is issued, indicating that the SDR could not be mapped. This helps identify potential gaps or limitations in the semantic comparison.

The purpose of comparing *comparisonText* with *subsequenceText* is to evaluate semantic or structural similarity, even when the texts are not identical. Since *subsequenceText* is a known part of *sourceText*, it acts as a reliable reference. By comparing *comparisonText* to this reference, we can assess how closely the new input aligns with known and meaningful segments of the original content. This design allows us to detect semantic closeness, even in cases where *comparisonText* is not an exact match but may convey a similar meaning.

```
// Merge all SDRs into single binary vectors
int[] sdrVector1 =
retrievedSDRs1.Values.SelectMany(sdr =>
sdr).ToArray();

int[] sdrVector2 =
retrievedSDRs2.Values.SelectMany(sdr =>
sdr).ToArray();
```

SDR values of each token id of subsequenceText are merged(concatenated) into one single binary vector and only the unique values are saved. The same process is followed for the comparisonText SDRs as well. The code snippet below explains how SDR indexes were merged:

This merging(concatenation) is done so that we can calculate the distance and similarity of this two merged arrays. sdrVector1 represents subsequenceText and sdrVector2 represents comparisonText.

E. Similarity Computation of Basic SDRs:

The merged sdrVector1(subsequenceText) sdrVector2(comparisonText) are compared using similarity functions:

Cosine Similarity: Measures the cosine of the angle between two SDR vectors, indicating how closely related the sequences are.

```
// Find the maximum length
int maxLength = Math.Max(vec1.Length, vec2.Length);

// Pad both vectors to the same length
int[] paddedVec1 = vec1.Concat(new int[maxLength -
vec1.Length]).ToArray();

int[] paddedVec2 = vec2.Concat(new int[maxLength -
vec2.Length]).ToArray();
```

Euclidean Distance: Calculates the direct spatial distance between two SDRs in high-dimensional space. Lower distance value means higher similarity.

If the vectors have different lengths, the shorter one is padded with zeros to match the longer vector before performing the distance calculation. Vector padding and the Euclidean distance is converted into similarity using the following code snippet:

```
// Compute Euclidean distance
double distance = Math.Sqrt(paddedVec1.Zip(paddedVec2,
(v1, v2) => (v1 - v2) * (v1 - v2)).Sum());

// Maximum possible distance (when vectors are completely
different)
double maxDistance = Math.Sqrt(maxLength);

// Convert distance to similarity (higher value means more
similarity)
return 1 - (distance / maxDistance);
```

F. SDR Generation Using HTM:

To process the input, encode sequences, and perform sequence learning and similarity analysis we used RunExperiment() function. It processed inputBits using the encoder according to the HTM configuration settings we set in the code and then it applied SDR encoding to sequences.

In this function, we measured execution time using Stopwatch to track how long the process runs. Then set up HTM's synaptic connections (Connections) based on a pre-defined configuration, created an HTM layer (CortexLayer) to process data and apply sequence learning. In the next step we initialized a Spatial Pooler with homeostatic plasticity control, which helps regulate neuron activity and maintain stability in HTM.

The **Spatial Pooler** is a crucial phase in HTM learning, responsible for processing every input provided by the user. It is designed to learn the SDR of a given binary input array, which is generated as an output from the Encoder. In essence, the Encoder and Spatial Pooler together act as a recording mechanism that captures and learns representations of incoming inputs. To effectively train the Spatial Pooler, we provide it with token-IDs as sequences. These token-IDs represent discrete elements of textual data that have been converted into numerical form. By feeding sequences of token-IDs into the Spatial Pooler, the system learns structured patterns and relationships between different parts of the input.

To monitor the learning process of the Spatial Pooler, we utilize the **Homeostatic Plasticity Controller.** This component ensures that the system maintains a balanced state during learning, preventing overfitting or underutilization of columns in the Spatial Pooler.

Once the Spatial Pooler is stable, the **Temporal Memory** algorithm is activated (layer1.HtmModules.Add("tm", tm);). The system processes each sequence separately, ensuring that learning is completed accurately. For every token in the sequence:

- The HTM layer computes the next state (layer1.Compute(input, true)).
- The active columns and corresponding cell SDRs are determined. The most reliable SDR representation (either ActiveCells or WinnerCells) is selected.
- The SDRs are stored to retain learned representations for future comparisons.

The figure given below summaries the overall process:

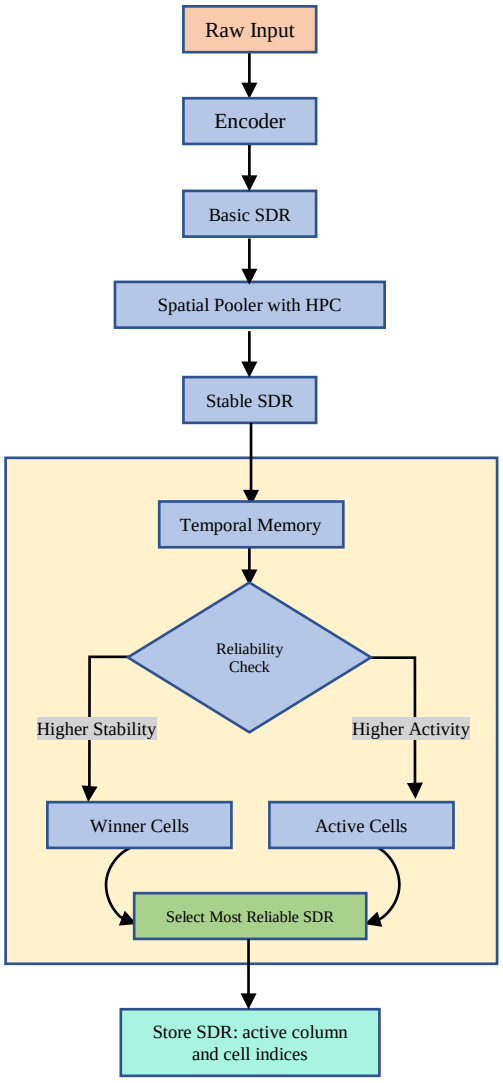

Figure 4: Storing Generated SDRs (from HTM) for processing

F.1. Merging SDRs for Subsequences:

Before computing similarity between subsequences, we merged the SDRs corresponding to each token ID within a given subsequence. This merging step is necessary because a subsequence typically contains multiple tokens, and as a result, each token ID generates its own individual SDR. However, our goal is to compare entire subsequences as unified entities, rather than performing token-by-token

comparisons. To achieve this, we combine the individual SDRs into a single representation—this process is referred to as merging.

The merging is analogous to concatenating words to form a sentence: just as words combine to express a complete thought, individual token SDRs are merged to capture the full meaning of a subsequence.

1. Retrieving and Converting Stored SDRs: The code snippet responsible for this-

```
List<int> tokens1 = listGeneration.TokenizeText(text1);
List<int> binarySDR1 = new List<int>();

 // Retrieve stored SDRs for each token and convert to binary
SDRs
foreach (var token in tokens1)
{
   var storedSDR= sdrStorage.LoadSDR(token.ToString());
    if (storedSDR.HasValue)
    {
      int[] columnBinary =ConvertToBinarySDR
      (storedSDR.Value.columnSDR, numColumns);

       binarySDR1.AddRange(columnBinary);

       // Convert cell SDR to binary
       int[] cellBinary = ConvertToBinarySDR
       (storedSDR.Value.cellSDR, numColumns *
       numCellsPerColumn);

       binarySDR1.AddRange(cellBinary);
    }
 }
```

We used two lists, binarySDR1 and binarySDR2, which would store the binary SDRs for two different sequences. It then iterates through tokens1, which contains tokenized representations of a given text. For each token, it retrieves its stored **SDR representation** from sdrStorage using LoadSDR(token.ToString()). If the SDR exists, it contains two primary components:

- **Column SDR**: Represents which columns were activated for this token. It represents the active columns in the HTM spatial pooler.
- **Cell SDR**: Represents the individual cells within active columns. It captures the temporal context within each column.

F.2. Converting SDRs to Binary Format

- The retrieved column SDR is converted to binary format using ConvertToDenseArray(), ensuring that active columns are represented with 1s in the correct positions.
- The retrieved cell SDR is similarly processed using ConvertToDenseArray(), mapping active cell locations to a binary vector.
- The generated binary representations are then stored in binarySDR1 and binarySDR2.

ConvertToDenseArray method takes a list of active indices (activeIndices) and the total size of the output SDR (totalSize). It initializes a binary vector of the given size (totalSize), setting all values to 0. It then iterates through the list of active indices and sets the corresponding positions in the binary vector to 1, ensuring that active locations are correctly represented. The resulting binary SDR is returned, maintaining the sparsity and structure of the original SDR.

```
private static int[] ConvertToDenseArray(int[]
activeIndices, int totalSize)
{
   var binarySDR = new int[totalSize];

   foreach (var index in activeIndices.Where(i => i >= 0
&& i< totalSize))
            {
               binarySDR[index] = 1;
            }

   return binarySDR;
}
```

G. Similarity Computation of Subsequence's SDRs:

We performed similarity comparisons between two binary SDRs (we got from merging in previous section) using two different metrics: **cosine similarity and Euclidean similarity.** First, it checked if both SDRs contain data (non-zero counts), then calculated and displayed both similarity scores. The cosine similarity measures the angle between vectors, ranging from 0 (opposite) to 1 (identical), while the Euclidean similarity converts the straight-line distance between vectors into a normalized score from 0 (dissimilar) to 1 (identical). If either SDR was empty, it output a message indicating no comparison could be made.

Cosine Similarity: For cosine similarity, the calculation involves computing the dot product of the vectors and dividing it by the product of their magnitudes, automatically returning zero if either vector's magnitude equals zero to avoid division errors.

Euclidean Distance: Euclidean similarity method aggregates the squared differences between corresponding vector elements, derives the Euclidean distance from this sum, and transforms it into a normalized similarity score through inversion. Both techniques incorporate safeguards against vector length mismatches by restricting comparisons to the minimum length between the two input vectors.

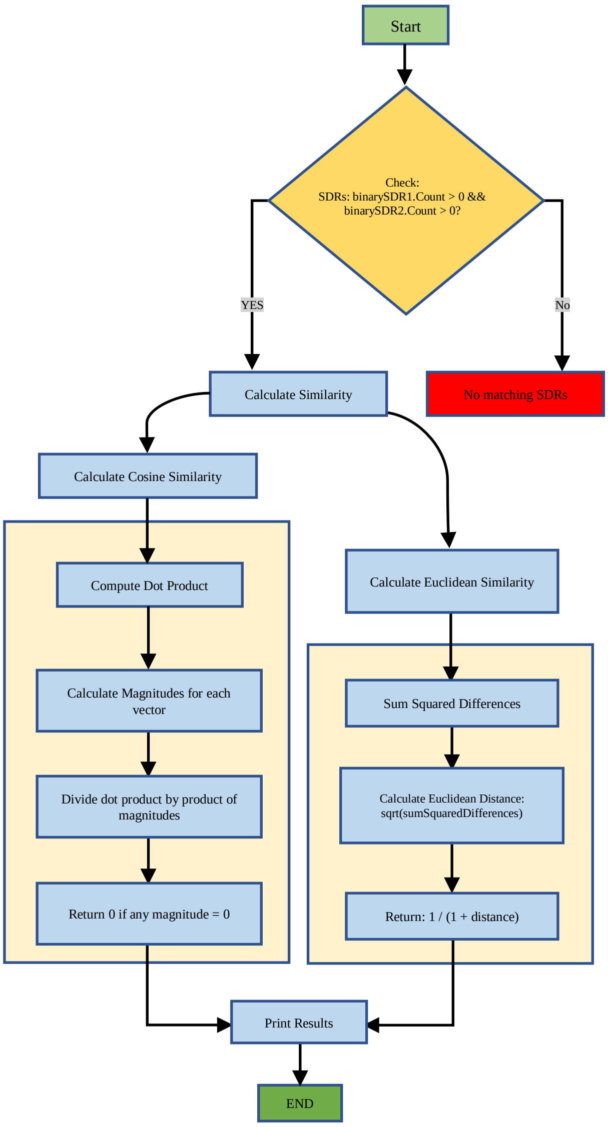

Figure 5: Workflow of Calculating Similarity between two subsequence-SDRs

IV. Results

The experiments conducted in this study aimed to evaluate the effectiveness of Sparse Distributed Representations (SDRs) in sequence learning and similarity comparison. The results are categorized into four key areas as already mentioned in the Methodology.

A. Tokenized Words:

Each word is assigned a unique token ID, which serves as the basis for Sparse Distributed Representation (SDR) generation via a Scalar Encoder. Since the Scalar Encoder is designed to process numerical values only, assigning token IDs to words enables their conversion into SDRs, ensuring compatibility with the encoding process.

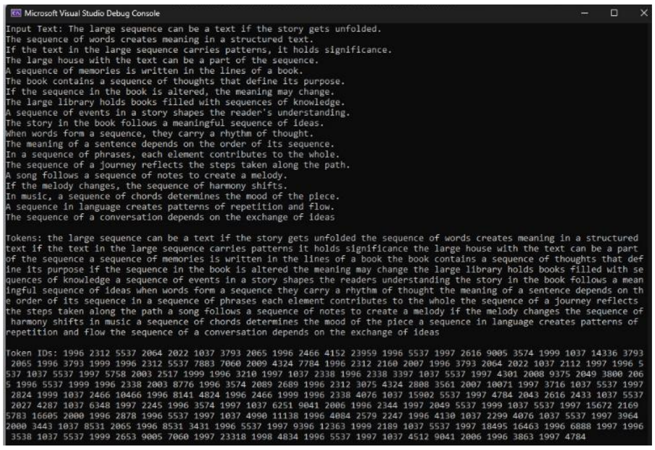

Figure. 6. Tokenized representation of sourceText

B. sourceText SDRs:

Using Scaler Encoder:

All the generated Token IDs in the previous step are converted to SDRs using Scalar Enocoder. Also the SDRs are saved to sdr_ouput.txt.

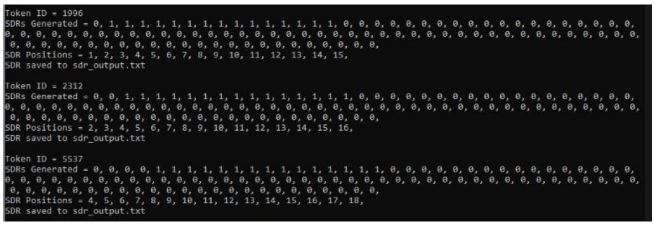

Figure. 7. SDR representation and SDR positions using Scaler Encoder

Using HTM:

For Column and Cell SDR generated using cortex layer (SP and TM), it was stored in sdr_data.json file. Each number represents the token_Id for tokens, Item1 represents column-SDR and Item2 represents cell-SDR for corresponding input token-ID. The SDRs here represents the index where the bit is 1. So, later we converted it to binary representation mentioned in ***F.2*** section in **Methodology**. A part of the file is attached here as:

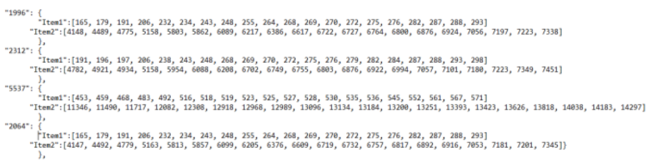

Figure. 8. Saved Active Column and Cell Indices for each token-ID

C. SDR mapping of subsequenceText and comparisonText:

Both the subsequenceText and comparisonText are taken as input from the user and converted to Token ID using BertTokenizer. Token IDs remain consistent throughout this process.

Our code will look for the SDR values of the corresponding Token IDs in the sdr_ouput.txt file (for basic SDRs) and sdr_data.json file (for HTM-SDRs).

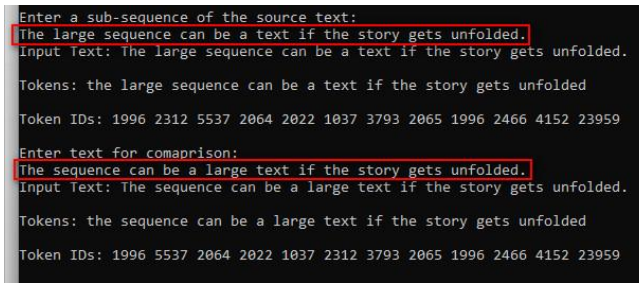

Figure. 9. Input subsequences converted into token-IDs

D. Measuring Similarity:

Using Basic SDRs:

Cosine Similarity and Euclidean Distance are computed between the merged SDR vectors. The resulting values usually range between 0 and 1, where higher values indicate greater similarity. But this values are converted to percentage for a better understanding.

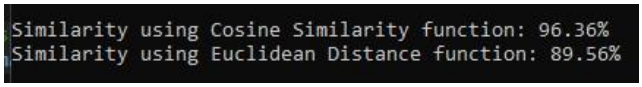

Figure. 10. Similarity Output

Using HTM SDRs:

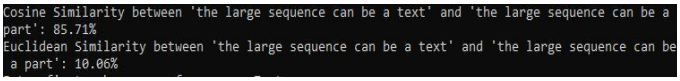

Figure. 11. Similarity Output of Subsequences

D. Result Analysis

In this section, we will discuss the differences in output similarity measurements. The table provided below illustrate that, for the same input subsequences, the similarity scores vary between direct SDRs and subsequences processed using HTM SDRs. This variation highlights the impact of HTM processing on similarity calculations, demonstrating how different encoding methods influence the final similarity score. Below is a table displaying the similarity scores for each pair of subsequences using both Cosine Similarity and Euclidean Similarity.

Table-1: Similarity Score Comparison for Input Subsequences

Input Subsequences	Semantic similarity score (using direct SDRs)	Semantic similarity score (using HTM SDRs)
"The large sequence can be a text" vs "the large sequence can be a part"	Cosine Similarity: 98.10% Euclidean Similarity : 92.44%	Cosine Similarity: 85.71% Euclidean Similarity : 10.06%
"The sequence of words creates meaning in a structured text." Vs "A structured sequence of words creates meaning in a text."	Cosine Similarity: 85.93% Euclidean Similarity : 79.45%	Cosine Similarity: 75.10% Euclidean Similarity : 30.06%
"When words form a sequence, they carry a rhythm of thought." Vs "The meaning of a sentence depends on the order of its sequence."	Cosine Similarity: 61.24% Euclidean Similarity : 75.51%	Cosine Similarity: 58.22% Euclidean Similarity : 33.58%

These results emphasize the differing impacts of direct SDR encoding and HTM processing on the representation of semantic relationships between subsequences. While direct SDRs maintain a high similarity score, HTM SDRs, which involve a more complex temporal processing mechanism, show varying degrees of similarity, especially when using Euclidean Similarity, indicating that HTM's representation may not always align as closely with direct SDRs for certain semantic constructs.

Both HTM and direct SDRs use sparse and distributed representations to encode information. However, HTM adds a layer of temporal context to these sparse representations. This makes HTM's SDRs not only sparse but also dynamic, evolving over time as new sequences are learned. As a result, HTM-processed SDRs can encode more meaningful relationships between elements in a sequence, accounting for the order and timing of events. In contrast, direct SDRs may only encode static relationships, which could limit their ability to capture the full richness of sequential data.

V. CONCLUSION

This study explored the application of Hierarchical Temporal Memory (HTM) and Sparse Distributed Representations (SDRs) for multi-sequence learning, focusing on tokenization, SDR generation, and similarity computation. By leveraging cosine similarity and Euclidean distance, we were able to analyze relationships between different sequences and assess how well the system identifies structured patterns. The results demonstrated that SDR-based representations effectively captured sequence relationships, providing a robust framework for temporal pattern recognition.

However, the current approach was limited to using only smaller iteration cycle for learning the pattern in the sequences that generates HTM SDRs that later are used for similarity computation. Future work should focus on to compute Cosine & Euclidean Similarity on 1536-dimensional vectors to assess high-dimensional representations, analyze individual dimensions to determine if certain dimensions contribute more to similarity than others and compare scalar values across dimensions to identify potential biases in similarity measurements.

By integrating these improvements, we can enhance the accuracy and robustness of HTM-based models, leading to deeper insights into sequence relationships and more reliable real-world applications. This integrated approach could provide deeper insights into sequence relationships and improve the performance of HTM-based models in real-world applications. Future experiments should explore this direction to achieve more accurate and reliable results.

VI. References

[1] ddobric/neocortexapi https://github.com/ddobric/neocortexapi

[2] Hawkins, J., & Ahmad, S. (2016). "Why Neurons Have Thousands of Synapses, a Theory of Sequence Memory in Neocortex.", Frontiers in Neural Circuits, 10, 23. 30 March 2016

[3] Ahmad, S., & Hawkins, J. "How do neurons operate on sparse distributed representations? A mathematical theory of sparsity, neurons and active dendrites." arXiv:1601.00720v2. 2016

[4] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." arXiv:1810.04805v2. 24 May 2019

[5] BertTokenizer https://huggingface.co/docs/transformers/model_doc/bert#transformers.BertTokenizer

[6] ML20/21-5.8. Implement SDR Representation Samples
Junaid Khalid, Abbad Zafar, Sabeen Javaid, Farrukh Shafique
https://github.com/ddobric/neocortexapi/blob/master/source/Documentation/ML2021-5.8.%20Implement%20SDR%20Representation%20Samples.pdf